hashlock.

Security Audit

Defi Bull World (DeFi)

March 2025 info@hashlock.com.au Hashlock Pty Ltd

Table of Contents

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

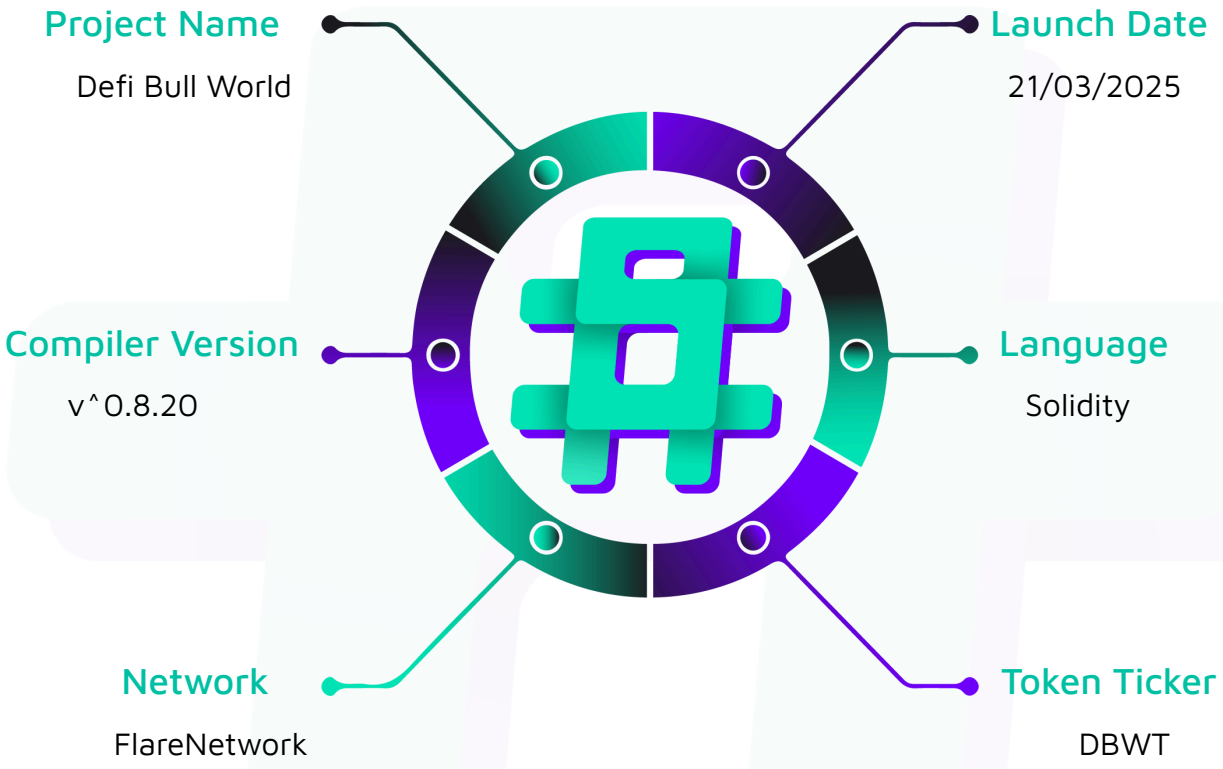
The Defi Bull World team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

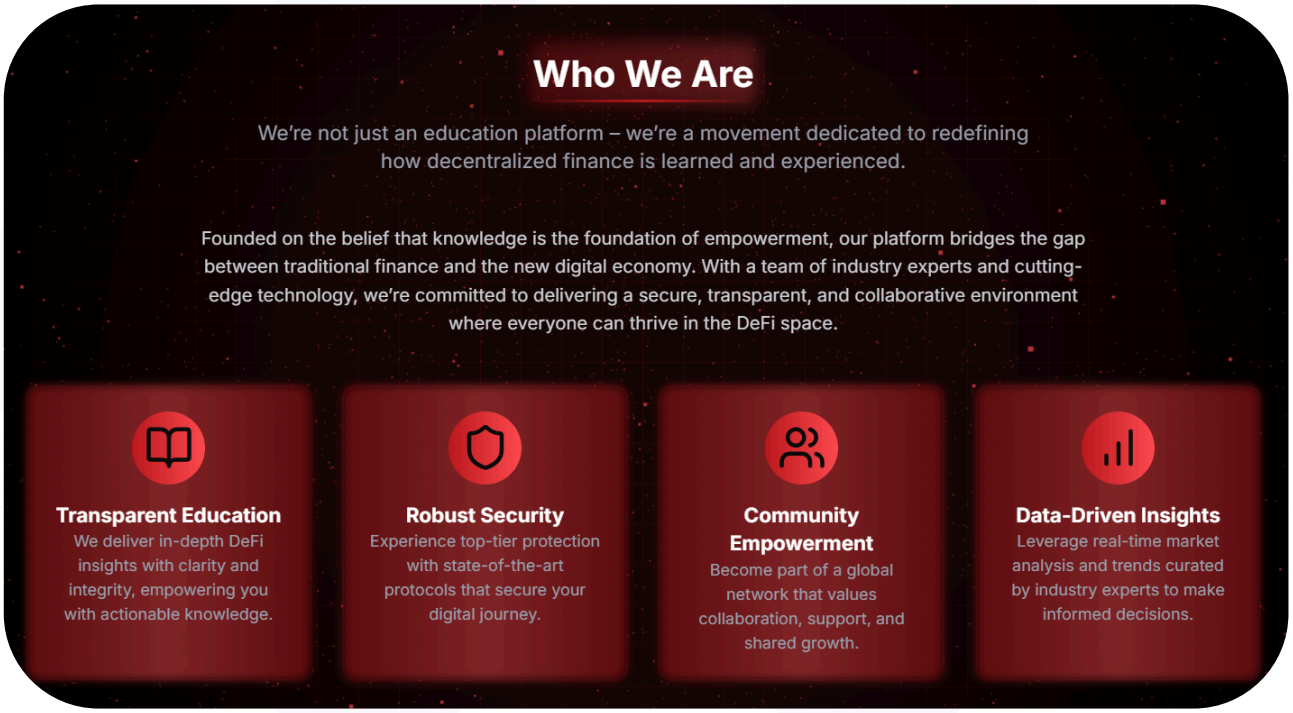
Defi Bull World is a cutting-edge digital asset and DeFi education platform built on the Flare Network with its native token FLR, led by seasoned blockchain experts since Bitcoin's inception. The platform delivers expert insights and regular updates through courses, webinars, and multimedia content, with plans to go multichain by Q3 2025. With new courses added monthly, AI-powered multilingual content, interactive experiences like sweepstakes and tests, and guaranteed long-term access for early members, Defi Bull World is committed to transparency through regular audits and public disclosure of contracts and wallets, while reinvesting revenues to drive sustainable growth.

Project Name: Defi Bull World
Compiler Version: ^0.8.20
Website: https://www.defibullworld.com/
Logo:

Visualised Context:



Project Visuals:



Audit Scope

We at Hashlock audited the solidity code within the Defi Bull World project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	**Defi Bull World Protocol Smart Contracts**
Platform	**Ethereum / Solidity**
Audit Date	**March, 2025**
Contract 1	DBT.sol
Contract 1 MD5 Hash	38d7f0b399a4b221045576d88d1da84b
Contract 2	DBW.sol
Contract 2 MD5 Hash	afb7b741e0ee6bd22e499128a482a5e7
Contract 3	DBWF.sol
Contract 3 MD5 Hash	746993f4e83d9e978fe21f9d8e3d8f1e
Contract 4	DBWL.sol
Contract 4 MD5 Hash	f86e7b84e2b4d705cccf0dae46dad400
Contract 5	NFTC.sol
Contract 5 MD5 Hash	4db077e4ff8359b852789df129e5599c
Contract 6	MembershipNFTCore.sol
Contract 6 MD5 Hash	bc85ddff99f843c8c5256b5282ef7954
Contract 7	MembershipPaymentManager.sol:
Contract 7 MD5 Hash	62650e26cab89dca9038dd2e0032d9d6
Contract 8	MembershipRewardsManager.sol:
Contract 8 MD5 Hash	31b7c1d46e1680850fab557711843eaf

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts. We initially identified some vulnerabilities that have since been addressed.

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the Audit Findings section. The list of audited assets is presented in the Audit Scope section and the project's contract functionality is presented in the Intended Smart Contract Functions section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

5 Medium severity vulnerabilities

2 Low severity vulnerability

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
DBT.sol - Allows users to: - Transfer tokens to other users with automatic burn mechanism - View current burn rate and total burned tokens - Check remaining burn allowance before max limit is reached - Allows admins to: - Set the burn rate for all transfers - Pause and unpause all token transfers - Upgrade the contract implementation - Initialize with full token supply (100 million tokens)	**Contract achieves this functionality.**
DBW.sol - Allows users to: - Transfer Defi Bull World WORLD tokens with automatic burn mechanism - Check their token balance and transfer history - View total burned tokens and remaining burn capacity - Allows admins to: - Adjust the burn rate applied to all transfers - Pause token transfers during emergencies - Unpause transfers to resume normal operations - Upgrade the contract to new implementations	**Contract achieves this functionality.**

DBWF.sol - Allows users to: - Transfer FTSO-related tokens with automatic burning - View token statistics including burn rate and total burned - Monitor when maximum burn limit is approaching - Receive tokens from other users - Allows admins to: - Change the burn rate percentage for all transfers - Temporarily halt all token transfers through pause mechanism - Resume token operations after pausing - Implement contract upgrades for new features	**Contract achieves this functionality.**
DBWL.sol - Allows users to: - Transfer Defi Bull World WORLD LIQUIDATOR tokens - Experience automatic burning on each transfer (deflationary mechanism) - View personal token balance and transaction history - Check the remaining tokens that can be burned - Allows admins to: - Configure burn rate for transfers - Pause all transfers in case of emergency - Unpause the contract to resume operations - Upgrade the contract functionality over time	**Contract achieves this functionality.**
NFTC.sol	**Contract achieves this**

- Allows users to: - Transfer NFT COLLECTION tokens with built-in burn mechanism - Monitor their token holdings and transfer activity - See the current burn rate applied to transfers - View burning statistics across the ecosystem - Allows admins to: - Set the desired burn rate for all token transfers - Pause token transfers if security issues arise - Unpause the contract when safe to resume operations - Upgrade the implementation to add new features or fix bugs	**functionality.**
MembershipNFTCore.sol - Allows users to: - Mint NFTs using native currency with optional referrals - Mint NFTs using ERC20 tokens (USDT or USDC) - Transfer NFTs to other users - View their owned NFTs across different tiers - Check their highest tier ownership and token balances - View detailed information about specific token IDs - See mint timestamps and token ownership history - Allows admins to: - Set payment and rewards manager contracts - Configure tier-specific URIs for metadata - Pause and unpause all contract functions during	**Contract achieves this functionality.**

emergencies - Withdraw ETH from the contract - Upgrade the contract implementation - Monitor tier supply and overall NFT distribution	
MembershipPaymentManager.sol: - Allows users to: - Make payments using native FLR tokens - Make payments using ERC20 tokens (USDT or USDC-E) - Use referrals when making payments - View their last payment details and timestamps - Check current prices for different membership tiers - Track referral status and rewards - View relationships with referrers - Allows admins to: - Set and update the FTSO price feed oracle - Configure slippage percentage for payments - Set USD prices for each membership tier - Update wallet addresses for revenue distribution - Configure stablecoin token addresses - Pause and unpause all contract functions - Withdraw ERC20 tokens from the contract - Upgrade the contract implementation	**Contract achieves this functionality.**
MembershipRewardsManager.sol: - Allows users to: - Claim rewards for specific token types - Claim all token rewards at once across all types - View their unclaimed rewards by token type - Check their total received rewards historically	**Contract achieves this functionality.**

- View all unclaimed rewards across all tiers - Track reward distribution details and history - Allows admins to: - Set token addresses for all five reward tokens (DBW, DBT, DBWF, DBWL, NFTC) - Configure reward amounts for each tier and token type - Record token rewards for users (shared with NFT Core contract) - Distribute token rewards directly to specific users - Set the authorized NFT Core contract address - Pause and unpause all contract functions - Upgrade the contract implementation	

Code Quality

This audit scope involves the smart contracts of the Defi Bull World project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was required.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Defi Bull World project smart contract code in the form of GitHub access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.
Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

Medium

[M-01] DBWToken&DBWFToken&DBWLToken&NFTCToken#nonReentrant - Inconsistent Reentrancy Protection Across Token Contracts

Description

Only the `DBTToken` contract implements `ReentrancyGuardUpgradeable` and applies the `nonReentrant` modifier to its administrative functions, while the other four contracts (`DBWToken`, `DBWFToken`, `DBWLToken`, and `NFTCToken`) lack this protection despite implementing identical functionality.

Impact

While the current implementations of the administrative functions do not make external calls that would enable reentrancy attacks, this inconsistency creates significant technical debt and potential security risks. Any future upgrades that add external calls to these functions would introduce reentrancy vulnerabilities in all contracts except DBTToken.

Recommendation

Add the `ReentrancyGuardUpgradeable` inheritance and nonReentrant modifiers to all administrative functions in `DBWToken`, `DBWFToken`, `DBWLToken`, and `NFTCToken`:

Status

Resolved

[M-02]DBTToken&DBWToken&DBWFToken&DBWLToken&NFTCToken#setBurnRate - Unbounded Burn Rate Vulnerability

Description

The `setBurnRate` function allows the owner to set an arbitrarily high burn rate without any upper limit. This could enable a malicious or compromised owner to effectively freeze token transfers by setting an extremely high burn percentage, circumventing the explicit pause functionality and potentially rendering the tokens unusable.

Vulnerability Details

All five token contracts implement the `setBurnRate` function that allows the owner to set any uint16 value as the burn rate without validation:

```
function setBurnRate(uint16 newRate) external virtual onlyOwner {

    uint16 oldRate = _burnRate;

    _burnRate = newRate;

    emit BurnRateUpdated(oldRate, newRate);

}
```

The burn rate is then used in the `_update` function to calculate the burn amount for each transfer:

```
uint256 burnAmount = (amount * _burnRate) / BURN_RATE_PRECISION;
```

Since `_burnRate` is a uint16 with a maximum value of 65,535 and `BURN_RATE_PRECISION` is set to 100,000, the owner could set a burn rate that would cause up to 65.535% of tokens to be burned during each transfer.

Impact

Normal use case: The burn rate is set to a reasonable value (e.g., 1 for 0.001%)

Exploit case: The burn rate is set to 65,535, resulting in 65.535% of tokens being burned on every transfer

Recommendation

Implement an upper bound for the burn rate by adding a constant and a requirement check:

```
function setBurnRate(uint16 newRate) external virtual onlyOwner {

    require(newRate <= MAX_BURN_RATE, "Burn rate exceeds maximum allowed");
    uint16 oldRate = _burnRate;

    _burnRate = newRate;

    emit BurnRateUpdated(oldRate, newRate);

}
```

This would limit the burn rate to a reasonable maximum (in this example, 10%) and prevent the function from being abused to effectively freeze token transfers.w

Status

Resolved

[M-03] DBTToken&DBWToken&DBWFToken&DBWLToken&NFTCToken - Lack of Asset Recovery Mechanisms

Description

All five token contracts inherit from ERC20Upgradeable and implement token functionality, but none include mechanisms to recover ETH or other ERC20 tokens that might be inadvertently sent to the contract address. Specifically:

The contracts lack functions to withdraw accidentally sent ETH

The contracts lack functions to withdraw accidentally sent ERC20 tokens (other than the native token of each contract)

This is problematic because:

Users occasionally make mistakes when sending transactions, Contracts may receive tokens through airdrops, Some protocols might send tokens to the contract address as part of their functionality.

In token migration scenarios, users might mistakenly transfer tokens to the contract

Once tokens are sent to these contracts, they become permanently locked without a recovery mechanism.

Impact

The impact of this vulnerability is directly proportional to the value of assets that might be accidentally sent to the contract

Users who accidentally send tokens to the contract address will permanently lose those assets. Lack of recovery mechanisms can lead to user frustration and reduced confidence in the protocol. If significant value is trapped in the contracts, this could affect market perception and token value.

Recommendation

Implement recovery functions for both ETH and ERC20 tokens with appropriate access controls:

```
// Add import for SafeERC20

import "@openzeppelin/contracts-upgradeable/token/ERC20/utils/SafeERC20Upgradeable.sol";


// Add in the contract

using SafeERC20Upgradeable for IERC20Upgradeable;


// Add events for transparency

event ERC20Recovered(address indexed token, address indexed to, uint256 amount);

event ETHRecovered(address indexed to, uint256 amount);
```

```
/**
 * @notice Allows the owner to recover ERC20 tokens sent to the contract by mistake
 * @param tokenAddress Address of the ERC20 token
 * @param amount Amount of tokens to recover
 */
function recoverERC20(address tokenAddress, uint256 amount) external onlyOwner nonReentrant {
    // Prevent recovering the contract's own token
    require(tokenAddress != address(this), "Cannot recover the contract's own token");

    IERC20Upgradeable token = IERC20Upgradeable(tokenAddress);
    uint256 balance = token.balanceOf(address(this));
    require(amount <= balance, "Insufficient token balance");

    token.safeTransfer(owner(), amount);

    emit ERC20Recovered(tokenAddress, owner(), amount);
}

/**
 * @notice Allows the owner to recover ETH sent to the contract by mistake
 * @param amount Amount of ETH to recover
 */
function recoverETH(uint256 amount) external onlyOwner nonReentrant {
    require(address(this).balance >= amount, "Insufficient ETH balance");
```

```
    (bool success, ) = payable(owner()).call{value: amount}("");
    require(success, "ETH transfer failed");

    emit ETHRecovered(owner(), amount);
}


// Add receive function to accept ETH transfers
receive() external payable {}
```

Status

Resolved

[M-04] MembershipPaymentManager#getFlareAmount - Oracle Price Data Timestamp Validation Vulnerability

Description

The contract retrieves price information from the Flare Time Series Oracle (FTSO) but fails to validate the timestamp of the returned data, potentially allowing stale price information to be used in critical payment calculations.

Vulnerability Details

The `getFlareAmount` function in the `MembershipPaymentManager` contract retrieves price data from the FTSO oracle but discards the timestamp information:

```
function getFlareAmount(uint256 usdPrice) public view returns (uint256) {
    // Get current price from FTSO
    (uint256 flarePrice, , ) = ftso.getCurrentPriceWithDecimals();
```

```
    // usdPrice comes in cents (2 decimals), convert to USD and calculate FLR amount
    return (usdPrice * 1e18 * 1e5) / (flarePrice * 100);
}
```

The function calls `ftso.getCurrentPriceWithDecimals()` which returns three values:

`_price` - captured as `flarePrice`

`_timestamp` - ignored (comma placeholder)

`_decimals` - ignored (comma placeholder)

By ignoring the timestamp, the contract cannot determine whether the price data is recent or stale, leading to a situation where outdated prices might be used for processing payments.

Impact

In volatile markets, stale oracle data could lead to incorrect pricing, allowing users to pay significantly less or more than the intended amount.

Recommendation

Modify the `getFlareAmount` function to retrieve and validate the timestamp:

```
// Add a constant for maximum allowed age of price data
uint256 public constant MAX_PRICE_AGE = 1 hours;

function getFlareAmount(uint256 usdPrice) public view returns (uint256) {
    // Get current price from FTSO including timestamp
    (uint256 flarePrice, uint256 timestamp, ) = ftso.getCurrentPriceWithDecimals();
```

```
    // Validate the price data is recent
    require(block.timestamp - timestamp <= MAX_PRICE_AGE, "Price data too old");

    // Calculate and return the FLR amount
    return (usdPrice * 1e18 * 1e5) / (flarePrice * 100);
}
```

Status

Resolved

[M-05] MembershipRewardsManager#recordTokenRewards - Duplicate Reward Recording

Description

The `recordTokenRewards` function lacks mechanisms to prevent duplicate reward recording. This vulnerability could allow users to receive multiple rewards for the same event, potentially draining the contract's token reserves and disrupting the project's tokenomics.

Vulnerability Details

```
function recordTokenRewards(
    address to,
    uint256 tier,
    uint256 amount
) external onlyNFTCoreOrOwner whenNotPaused validTier(tier) {
    // Record rewards for each token type if reward amount > 0
    for (uint256 tokenType = 0; tokenType <= NFTC_TOKEN; tokenType++) {
        if (tierTokenRewards[tier][tokenType] > 0) {
```

```
            uint256 tokenAmount = tierTokenRewards[tier][tokenType] * amount;

            unclaimedRewards[to][tokenType][tier] += tokenAmount;

            emit TokenRewardRecorded(to, tier, tokenType, tokenAmount);

        }

    }

}
```

The function:

Impact

If the NFT Core contract calls this function multiple times for the same event, users could receive duplicate rewards, potentially draining the contract's token reserves.

Recommendation

Option 1: Implement a Rewards Registry

Create a mapping to track which NFT minting events have already been rewarded:

```
// Add a mapping to track rewarded mint events

mapping(address => mapping(uint256 => mapping(uint256 => bool))) private rewardedMints;


function recordTokenRewards(

    address to,

    uint256 tier,

    uint256 amount,

    uint256 mintId  // Add a unique identifier for the mint event

) external onlyNFTCoreOrOwner whenNotPaused validTier(tier) {

    // Check if this mint has already been rewarded

    require(!rewardedMints[to][tier][mintId], "Rewards already recorded for this mint");
```

```
    // Mark this mint as rewarded
    rewardedMints[to][tier][mintId] = true;

    // Record rewards as before
    for (uint256 tokenType = 0; tokenType <= NFTC_TOKEN; tokenType++) {
        if (tierTokenRewards[tier][tokenType] > 0) {
            uint256 tokenAmount = tierTokenRewards[tier][tokenType] * amount;
            unclaimedRewards[to][tokenType][tier] += tokenAmount;
            emit TokenRewardRecorded(to, tier, tokenType, tokenAmount);
        }
    }
}
```

Option 2: Track Rewards Per NFT Token ID

If NFTs have unique token IDs, use these to track reward status:

```
// Add a mapping to track which NFT token IDs have been rewarded
mapping(uint256 => bool) private rewardedTokenIds;

function recordTokenRewardsForNFT(
    address to,
    uint256 tier,
    uint256 tokenId
) external onlyNFTCoreOrOwner whenNotPaused validTier(tier) {
    // Check if this token ID has already been rewarded
```

```
    require(!rewardedTokenIds[tokenId], "Rewards already recorded for this token");

    // Mark this token as rewarded

    rewardedTokenIds[tokenId] = true;



    // Record rewards as before (with amount=1 for a single NFT)

    for (uint256 tokenType = 0; tokenType <= NFTC_TOKEN; tokenType++) {

        if (tierTokenRewards[tier][tokenType] > 0) {

            uint256 tokenAmount = tierTokenRewards[tier][tokenType];

            unclaimedRewards[to][tokenType][tier] += tokenAmount;

            emit TokenRewardRecorded(to, tier, tokenType, tokenAmount);

        }

    }

}
```

Option 3: Implement Transaction Nonce Pattern

Use a nonce-based approach to prevent replay attacks:

```
// Add a mapping to track the next nonce for each user

mapping(address => uint256) private userRewardNonces;



function recordTokenRewards(

    address to,

    uint256 tier,

    uint256 amount,

    uint256 nonce

) external onlyNFTCoreOrOwner whenNotPaused validTier(tier) {
```

```
    // Check that the provided nonce matches the expected next nonce
    require(nonce == userRewardNonces[to], "Invalid nonce");

    // Increment the nonce
    userRewardNonces[to]++;

    // Record rewards as before
    for (uint256 tokenType = 0; tokenType <= NFTC_TOKEN; tokenType++) {
        if (tierTokenRewards[tier][tokenType] > 0) {
            uint256 tokenAmount = tierTokenRewards[tier][tokenType] * amount;
            unclaimedRewards[to][tokenType][tier] += tokenAmount;
            emit TokenRewardRecorded(to, tier, tokenType, tokenAmount);
        }
    }
}
```

Status

Resolved

Low

[L-01]DBTToken&DBWToken&DBWFToken&DBWLToken&NFTCToken#initializer - Redundant Initialization Protection

Description

All five token contracts inherit from `TokenStorageV1`, which defines a boolean `_initialized` flag. The `initialize` function in each contract uses both the OpenZeppelin initializer modifier and a manual check of this custom flag:

```
function initialize(address initialOwner) initializer public {

    require(!_initialized, "Contract already initialized");



    // Initialization logic...



    _initialized = true;

}
```

Impact

The redundant checks add unnecessary complexity without providing additional security. Having multiple mechanisms for the same purpose makes the code harder to maintain, as changes to initialization logic must consider both mechanisms.

Recommendation

Remove the custom `_initialized` flag check from the initialize function:

Status

Resolved

[L-02] MembershipCore#_removeTokenFromUserTier, safeTransferFrom - Inefficient array length access in the loop

Description

The `_removeTokenFromUserTier` function reads the length of the `tierTokens` array from storage during every iteration of a for a loop. This practice is inefficient because accessing storage is significantly more expensive in terms of gas costs compared to accessing memory. Each time the array length is read from storage, it incurs additional gas costs, which can accumulate and make the function unnecessarily expensive to execute, especially for large arrays. The same issue of reading `fromUniqueIds.length` multiple times in the `safeTransferFrom` function.

Recommendation

Cache the array length before the loop and use it in the loop.

Status

Resolved

Centralisation

The Defi Bull World project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.

Conclusion

After Hashlock's analysis, the Defi Bull World project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au
Contact: info@hashlock.com.au